Aarhus University

Department Of Computer Science

Compilation

Hand-in for Phase 4 (Individual Report)

Author
Søren M. Damsgaard

Student number
202309814

November 5, 2026



Task 1 - The Parser

The lexer didn't need any changes, so all the work happened in `parser.mly`. The entry rule `prog` now parses a `nonempty_list` of `fundecl`, where each function is a return type (`rettyp`, either `void` or a normal type), a name, a comma separated list of `param` (`name : type`) and a list of statements as the body. A new rule `RETURN; SEMICOLON` was added so `return;` gives a `ReturnStm` with `ret = None`.

Comma expressions got their own rule `expr_with_comma`, which is left recursive so `a, b, c` is parsed as `(a, b), c`. It's used everywhere a "full" expression is allowed (conditions, return, the for-loop init/update, expression statements and inside parentheses), while function call arguments still use the plain `expr` rule. That way a comma inside a function call is always an argument separator, and a comma expression has to be wrapped in parentheses to be used as an argument.

Task 2 - The Semantic's

The type checking in `semant.ml` is done in two passes, as suggested in the assignment, so that functions can call/use each other no matter the order they're declared in.

1. **Pass 1** (`check_signatures`) goes through all the function signatures only. It checks for duplicate parameter names, duplicate function names (also against the built-in library functions) and that there is exactly one `main` with the signature `int main()`. The valid signatures are then put into the function environment together with the library functions.

2. **Pass 2** (`typecheck_fundecl`) checks each function body with that environment. Every function gets a fresh scope where the parameters are added as normal local variables (so they can be modified), and the return type is saved in the environment so every `return` can be checked against it. Non-void functions are also checked so all paths end in a return.

For comma expressions we followed the type settings $\tau(e_1, e_2) \implies \tau(e_2)$ where $\tau(e_1)$ is discarded sicne it's only used for it's side-effects.

The `TypedAst` was extended with `Param`, `FunTyp` and `fundecl`, and `errors.ml` got a handful of new errors to accomodate errors when checking singatures in pass 1 and funciton bodies in pass 2

Task 3 - Code Generation

Each Dolphin function is translated into a LLVM function by `cgen_fundecl`, and `codegen_prog` just maps it. Every parameter becomes an LLVM argument, which is copied into a memory/stack slot with `alloca` + `store` at the start of the function, like in the example from the assignment. The stack/memory slot is then put into the codegen environment, so the function can treat the parameters as local variables.

Our functions get's a prefix `dolphin_fun_` and the `main` becomes `dolphin_main` (which is what `runtime.c` calls), so our functions can't clash with the C or library functions with the same name.

A comma expression generates the code for the left side first, ignores its value, then generates the right side and uses its value as the result.

Task 4 - Testing

All the old tests from phase 1–3 were wrapped in `int main() { ... }` so they work with the new syntax, and the old JSON test files were removed and new ones added to cover most of the new functionality.
Like in phase 3, all the tests are run by `test_dolphin.ml` with `OUnit2`, which checks the printed output and exit code of positive tests, and the location and message of the error for negative tests, all the good shiz.

Task 5

Glory Task - Not Implemented

How to run the code

You will have received a zip file containing the complete OCaml compiler source code, the Dune build configurations, and the C runtime environment.

This is run/compiled in VScode using the docker container provided.
To compile and run the project inside a VS Code development container, execute the following commands in the terminal while inside the folder:
The following commands is to run a test file manually

1. `dune build`
2. `dune exec dolphin -- compile --phase 4 assignment-06-tests/phase4/[name of test].dlp -o out.ll` (This compiles a test and outputs it into a file out.ll, prints errors in the terminal if any.)

3. `clang runtime.c out.ll -o ret`
4. `./ret` (outputs any prints into the terminal, like 'print_integer()')
5. `echo $?` (Outputs the result of the test)

The following commands is to run all the test suites using `OUnit2`

1. `dune build` (if not already done)
2. `dune test` (runs the 'test_dolphin.ml' file)

The test file asserts each test case with the expected output, be it error messages or numerical output.